PAYSTACK AI BUILDER CHALLENGE 2026

Frontend Implementation Plan & UX Design Spec

DisputePilot + TrustLayer
Merchant app, public trust profile and embeddable widget

Prepared for: Ashburg (Frontend)
Based on: Frontend Role Guide, Paystack Architecture doc, Project Path spec, UI mockup
Version 1.0 - 19 September 2026

Contents

1. Executive summary . . . 2
2. Review: what is solid, what is missing . . . 3
3. Language and technology stack . . . 5
4. Project structure . . . 7
5. UX direction and design system . . . 8
6. Information architecture and flows . . . 12
7. Screen-by-screen specification . . . 13
8. TrustLayer on the customer end . . . 19
9. Data layer, auth and realtime . . . 24
10. Quality: states, accessibility, testing . . . 26
11. Schedule, scope and risks . . . 28
12. Figma handoff kit . . . 29

1. Executive summary

You are building three surfaces on top of one backend: a private **merchant app** (DisputePilot), a public **TrustLayer profile** and an **embeddable trust widget**. Your job, in the words of your role guide, is to turn backend facts into something merchants and customers can understand and act on. This document is the frontend plan, the UX design spec (written so it can be built in Figma 1:1), and the recommendation for how TrustLayer should appear to customers. The companion PDF lists exactly what you need from the backend.

The short answers

Question	Answer
Best language?	**TypeScript (strict mode)** end to end. The API contract, the money/time helpers and the widget payloads are where bugs hide in a payments product; TypeScript plus Zod runtime validation catches contract drift the moment the backend changes a field.
Framework?	**Next.js (App Router) + React**, Tailwind CSS, shadcn/ui, TanStack Query. This matches the Project Path spec and your existing stack. The widget is a separate **vanilla TypeScript** bundle (about 10KB), not React.
Native mobile app?	No. Merchants and customers are served by a responsive web app (installable as a PWA if time allows). A React Native app would double the work for no demo benefit; the design tokens and typed API client remain reusable later.
Can I use Figma?	Not from this chat: I have no Figma tool connected, so I cannot draw inside your Figma file. What I did instead: a complete, build-ready design spec (tokens, components, screen layouts, wireframes, states, prototype flows) in sections 5 to 8 and 12, structured so you or a designer can recreate it in Figma in a few hours. If a Figma connector is available to you, I can attempt to use it once connected.
Best way to show TrustLayer to customers?	A **domain-verified badge (iframe-based) that opens a trust card and links to a hosted public profile**, placed on merchant-controlled pages (product page, pre-payment page, order confirmation), plus a shareable link and QR code for merchants who sell on WhatsApp or Instagram. Full detail in section 8.

What I reviewed

- **Frontend Role & Implementation Guide**: your scope, screens, endpoints, build order, definition of done.
- **Paystack architecture / TrustLayer extension**: trust engine, signals, privacy rules, demo story, the caveat that Paystack-hosted checkout cannot be customised.
- **DisputePilot Project Path** (37 pages): schema, endpoints, dispute defense, reconciliation, AI prompts, 14-day plan and demo script.
- **Your mockup image** (4 screens + design system): treated as the visual baseline. I keep its look (navy sidebar, blue accent, Inter) and fix its gaps (section 2).

> **The one-sentence design idea**
> Do not build a collection of dashboards; build a guided workflow: understand the payment, identify the issue, gather evidence, act, then maintain operational trust. Every screen should end in a clear next action that leads to the next screen.

2. Review: what is solid, what is missing

Your mockup is a strong start: clear hierarchy, consistent cards, a sensible palette and a good instinct for an evidence checklist plus an AI draft. Compared with the guide and the spec, these gaps should be closed before or during the build. Items 1 to 4 are demo-critical.

#	Area	Mockup today	Spec / guide requires	Action
1	Dispute deadline	No countdown; only a 'Needs response' pill	Live countdown, critical banner under 8h, colour by remaining time (Project Path 8.1, 10.1)	Add countdown to dashboard banner, dispute list cards and dispute detail
2	Reconciliation	Sidebar item only	Core feature #1; demo story starts with an AI match being accepted	Design a lean screen: tabs, AI match card, sticky summary bar
3	Fulfillment logging	Not shown	Demo step 'merchant logs delivery proof'; readiness and TrustLayer depend on it	Quick-log modal + post-sale prompt card on dashboard
4	Submit control	'Submit to Paystack' sits beside 'Review Evidence'	AI evidence must be reviewable, never silently submitted	Gate submit behind review + confirmation modal + 3-step progress
5	TrustLayer states	Only 'Verified'	Four public states: Verified, Established, Building Trust, Not Yet Rated	Design all four, plus hide rules (section 8)
6	TrustLayer actions and history	Signals only	Recommended actions, 7/30/90-day history, reason codes	Add both panels to the merchant TrustLayer screen
7	Public copy	'Trusted Merchant', 'Reliable Support', 'Powered by trusted partners'	'No UI claims not supported by backend data'; trust is informational, not a guarantee	Reword to the four backed signals; drop 'Reliable Support' unless a support signal exists
8	Navigation bug	TrustLayer settings screen highlights 'Reconciliation' in the sidebar	Active item must match the route	Fix in Figma; derive active state from the route in code
9	Accept-and-refund path	Not shown	Resolve API supports merchant-accepted with refund amount	Secondary, confirmed, destructive action on dispute detail
10	Loading / empty / error / mobile	Public profile has a phone; merchant screens do not	Every API screen: loading, empty, error, success; desktop + mobile	Design the state matrix (section 10) and a mobile set
11	Embed snippet	Single script tag	Safe embedding on merchant sites	Script that mounts a sandboxed iframe; domain verification; fallbacks

Decisions to settle with the team (they change your code)

Topic	Conflict	Recommendation
Route names	Project Path uses '/' and '/reconciliation'; the guide uses '/dashboard', '/trustlayer/profile'	Use the guide's paths; make TrustLayer profile/widget tabs under '/trustlayer'. Public pages live under a separate public route group.
Score thresholds	Quality 'Strong' is 70+ in the FE section but 80+ in the AI section; readiness bands are 70 / 40	Ask the backend to return a `label` with every score. The UI never hard-codes thresholds.
API prefix	Project Path: base '/api/v1'; guide lists bare paths	Use '/api/v1' and proxy it through Next.js route handlers.
Merchant id in URLs	Trust endpoints take `:merchantId`	Authenticated calls should use `/trust/me` (id from the session). Only the public endpoint takes a slug. Avoids id-guessing bugs.
Dispute window	Spec assumes 16 hours	Confirm against Paystack's current docs. Build purely from `due_at`, never from a hard-coded 16.
Reconciliation priority	Guide: 'if time permits'; demo script: opens with it	Build a lean version (accept/reject AI match, complete) in the demo path. Skip session history and sparkline.
Dispute status vocabulary	Internal enum (pending, evidence_generated, submitted, won, lost, auto_accepted) vs Paystack values	Backend returns both `status` and `paystack_status`; UI maps internal status to labels.
Token storage	JWT access + refresh tokens	Keep tokens in httpOnly cookies via the Next.js proxy, not in localStorage (section 9).

3. Language and technology stack

Recommendation: TypeScript, Next.js, Tailwind

Use **TypeScript with** `strict: true` everywhere. Your profile already centres on TypeScript, Tailwind and React, the Project Path spec names Next.js, and the backend is Node/TypeScript-friendly, so you can share types or generate them from an OpenAPI file. The table lists each layer, the pick and why.

Layer	Pick	Why	Rejected
Language	TypeScript (strict)	Compile-time safety across API types, money and status enums	Plain JS
Framework	Next.js 14+ App Router, React	SSR/ISR for the public profile (fast, cacheable), client components for the app, route handlers as a proxy	Vite SPA (weak for the public profile SEO/perf)
Styling / UI kit	Tailwind CSS + shadcn/ui (Radix)	Fast to build; accessible primitives (dialog, popover, tabs); tokens map 1:1 from Figma	MUI/Chakra (heavier, harder to match your mockup)
Server state	TanStack Query v5	Caching, polling, retries, optimistic updates; per-screen loading/error states come almost free	Redux
UI state	Zustand (small)	Sidebar, quick-log modal, filters	Context for everything
Forms / validation	React Hook Form + Zod	Fast forms; the same Zod schemas validate API responses at runtime	Formik
Tables	TanStack Table	Sort/filter/paginate headless, styled with Tailwind	Ad-hoc tables
Charts	Recharts	Enough for revenue, readiness, trust history	D3 (overkill)
Dates / money	date-fns(-tz), Intl.NumberFormat	Africa/Lagos time; kobo (integer) to naira formatting	Floats, moment
Uploads	react-dropzone + signed URLs	Evidence photos and PDFs	Base64 in JSON
Realtime	Native WebSocket wrapper + polling fallback	Dispute alerts and countdown sync	Polling only
Mocking	MSW + fixtures	Build every screen before the backend exists (guide phase 6)	Hard-coded JSON in components
Testing	Vitest + Testing Library, Playwright	Unit for utils, component tests for states, one E2E for the demo path	Manual only
Widget	Vanilla TypeScript, built with Vite/tsup	About 10KB; must not drag React onto merchant sites	React widget
Hosting	Vercel (or any Node host)	Two projects: merchant app, public trust site (separate domain)	Single domain for both

Architecture rules

1. **One typed API client** (`lib/api.ts`). Components never call `fetch` directly. Every response is parsed with a Zod schema.
2. **Backend-for-frontend proxy.** The browser calls `/api/...` on your Next.js origin; a route handler forwards to the backend and attaches auth from an httpOnly cookie. This removes CORS pain, keeps tokens away from JavaScript, and means Paystack keys can never appear in the frontend.
3. **Server components for public pages**, client components for interactive app screens.
4. **Separate the public surface.** The public profile and widget are served from their own domain/project with no merchant cookies, strict caching and a tight Content-Security-Policy. A bug in the merchant app can never leak into the public site.

5. **No business logic in the browser.** Scores, labels, match confidence, readiness and trust states all arrive from the backend. The frontend formats and explains; it never decides.
6. **Components are reusable, not duplicated** (guide definition of done): one StatusBadge, one MoneyText, one CountdownTimer, one TrustBadge.

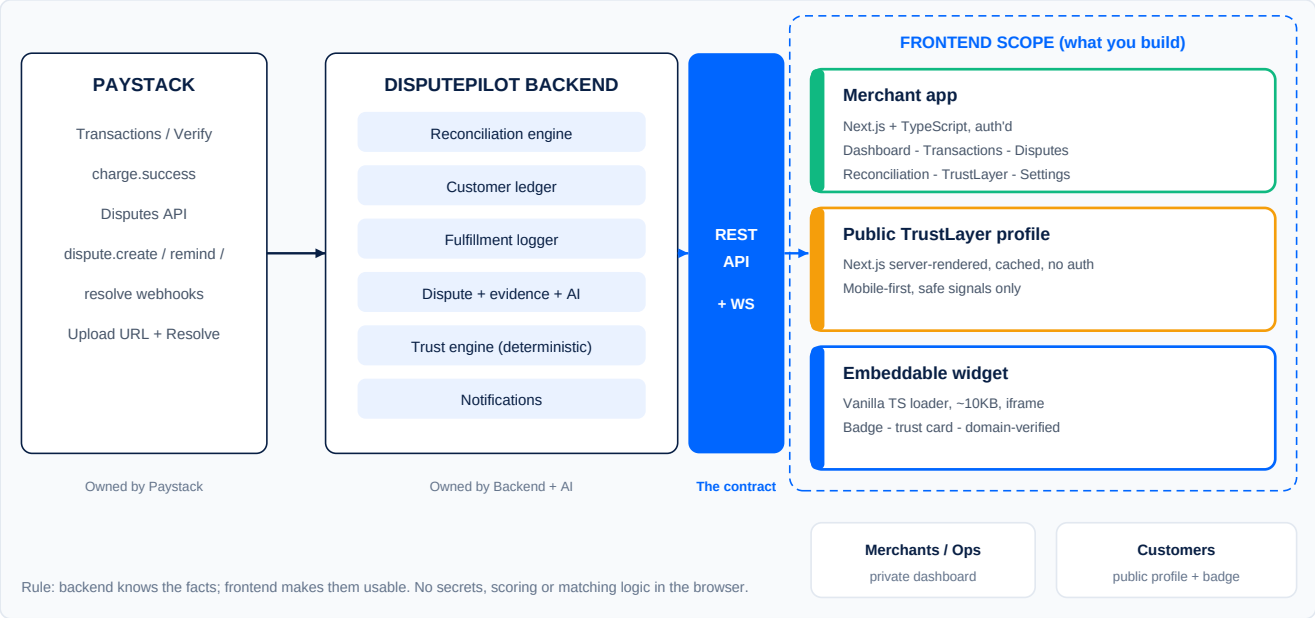


Figure 1. System position. The dashed box is your scope; the blue bar is the API contract you agree with the backend developer.

4. Project structure

The layout below extends the guide's suggested structure with route groups, a proxy, feature hooks, mocks and a separate widget package. In the guide, `trustlayer/profile` and `trustlayer/widget` are merchant-side pages; keep them as tabs of one TrustLayer screen and render the real public page from the `(public)` group.

```
disputepilot-web/
|-- src/
|   |-- app/
|   |   |-- (auth)/login/page.tsx
|   |   |-- (auth)/register/page.tsx           # + Paystack connect onboarding
|   |   |-- (app)/layout.tsx                   # AppShell: sidebar, topbar, auth guard
|   |   |-- (app)/dashboard/page.tsx
|   |   |-- (app)/transactions/page.tsx
|   |   |-- (app)/transactions/[reference]/page.tsx
|   |   |-- (app)/disputes/page.tsx
|   |   |-- (app)/disputes/[id]/page.tsx       # flagship screen
|   |   |-- (app)/reconciliation/page.tsx
|   |   |-- (app)/customers/page.tsx
|   |   |-- (app)/customers/[id]/page.tsx
|   |   |-- (app)/orders/page.tsx
|   |   |-- (app)/trustlayer/page.tsx          # tabs: Overview | Public profile | Widget
|   |   |-- (app)/settings/page.tsx
|   |   |-- (public)/t/[slug]/page.tsx         # public TrustLayer profile (SSR + ISR)
|   |   |-- (public)/embed/[slug]/page.tsx     # iframe target used by the widget
|   |   `-- api/[...path]/route.ts             # BFF proxy to backend /api/v1
|   |-- components/
|   |   |-- ui/            # shadcn primitives + Button, Badge, Card, Skeleton, EmptyState
|   |   |-- layout/        # AppShell, Sidebar, TopBar, MobileTabBar
|   |   |-- shared/        # MoneyText, PaystackRef, ChannelBadge, StatusBadge, CountdownTimer
|   |   |-- dashboard/  transactions/  disputes/  reconciliation/  fulfillment/
|   |   |-- trustlayer/    # TrustBadge, TrustCard, SignalList, TrustHistory, EmbedCode
|   |   |-- charts/  tables/
|   |-- features/          # hooks per domain: use-disputes.ts, use-trust.ts ...
|   |-- lib/               # api.ts, auth.ts, money.ts, time.ts, query-keys.ts, realtime.ts
|   |-- schemas/           # Zod schemas (source of truth for runtime validation)
|   |-- types/             # transaction.ts dispute.ts customer.ts trust.ts (z.infer)
|   `-- mocks/             # MSW handlers + fixtures: strong / incomplete / new merchant
|-- widget/                # separate package: vanilla TS loader + iframe bridge
|   |-- src/loader.ts
|   `-- vite.config.ts
|-- tests/e2e/demo-path.spec.ts
`-- tailwind.config.ts     # tokens from section 5
```

Naming and conventions

- Files: kebab-case; components: PascalCase; hooks: `useXyz`; one component per file; props typed with an interface.
- Money is always an **integer in minor units** (kobo) in types and state; convert only inside `MoneyText`.
- Timestamps are ISO-8601 UTC strings from the API; format in `Africa/Lagos` by default, merchant-configurable later.
- Query keys come from a single factory (`lib/query-keys.ts`), so invalidation is predictable.
- Feature flags for anything cut from the MVP (customers, session history) so the sidebar never links to a half-built page.

5. UX direction and design system

5.1 Design principles

Principle	What it means in the UI
Guided workflow	Every screen ends with the next best action: payment to dispute to evidence to trust. No dead ends.
Urgency is visible	Deadlines are big, live and colour-coded. A critical dispute cannot be missed or dismissed.
Evidence over assertion	Show exactly what exists and what is missing, with a suggestion for each gap. Never claim more than the data supports.
Calm trust	TrustLayer is quiet and factual: an icon, a status, a sentence, a date. No hype, no stars, no exact numbers.
Precise money	Amounts use tabular numerals, the right currency symbol and integer minor units. A wrong digit destroys trust in a payments tool.
AI assists, humans decide	AI content is labelled, editable and never submitted without an explicit review step.

5.2 Colour tokens

Base palette taken from your mockup (Navy, Blue, Green, Orange, Gray), extended with a danger red, surfaces and accessible text variants. The mockup's green and orange fail contrast as small text on white, so use the darker text variants for labels and keep the bright values for fills, icons and bars.

Token	Hex	Use
brand.navy (primary)	#081F44	Sidebar, headings, primary buttons on light surfaces
brand.blue (accent)	#0066FF	Links, active nav, primary action, AI/information accents
status.success	#10B981 (text #047857, tint #E7F8F1)	Success, verified, matched. Use #047857 for text on white or tint
status.warning	#F59E0B (text #92400E, tint #FEF3DC)	Missing evidence, moderate readiness, Building Trust
status.danger	#EF4444 (text #B91C1C, tint #FDECEC)	Critical dispute, failed, vulnerable, destructive actions
status.info	#0066FF (text #0052CC, tint #EAF2FF)	Evidence generated, AI suggestions, Established
neutral.gray	#64748B (text-secondary #475569)	Secondary text, disabled, Not Yet Rated
surface.bg / card / border	#F8FAFC / #FFFFFF / #E2E8F0	Page, cards, dividers
text.primary	#0F172A	Body text

5.3 Typography, spacing, shape, motion

Token	Value
Font	Inter (via next/font), fallback system-ui. Mono for references: ui-monospace / JetBrains Mono. Enable tabular numerals on all money and table numbers.
Scale (from mockup)	H1 32/40 semibold - H2 24/32 - H3 20/28 - Body 16/24 - Small 14/20. Dashboard tables and cards use 14px base, 12px for captions.
Spacing	4px base, 8pt rhythm: 4, 8, 12, 16, 24, 32, 48
Radius	6 inputs and buttons - 12 cards and modals - 999 pills and badges
Elevation	Cards: 1px border + very soft shadow. Popovers/modals: medium shadow. No heavy drop shadows.
Breakpoints	360 mobile - 768 tablet - 1024 laptop - 1440 desktop (design frames at 1440 and 390)
Motion	150-200ms ease-out for hover/expand. Countdown pulse only under 1 hour. Everything respects prefers-reduced-motion.

```
// tailwind.config.ts (excerpt)
theme: { extend: {
  colors: {
    navy: '#081F44', brand: '#0066FF',
    success: { DEFAULT: '#10B981', text: '#047857', tint: '#E7F8F1' },
    warning: { DEFAULT: '#F59E0B', text: '#92400E', tint: '#FEF3DC' },
    danger:  { DEFAULT: '#EF4444', text: '#B91C1C', tint: '#FDECEC' },
    info:    { DEFAULT: '#0066FF', text: '#0052CC', tint: '#EAF2FF' },
    surface: { bg: '#F8FAFC', card: '#FFFFFF', border: '#E2E8F0' },
  },
  borderRadius: { md: '6px', xl: '12px' },
  fontFamily: { sans: ['var(--font-inter)', 'system-ui'], mono: ['ui-monospace', 'monospace'] },
}}
```

5.4 Status semantics (one mapping, used everywhere)

Domain	Value	Label shown	Colour
Transaction	success / failed / abandoned / reversed	Successful / Failed / Abandoned / Reversed	green / red / gray / amber
Reconciliation	auto_matched / manually_matched / unmatched / flagged / ai_suggested	Matched (auto) / Matched / Unmatched / Flagged / AI suggestion (87%)	green / green / amber / red / blue
Dispute	pending / evidence_generated / submitted / won / lost / auto_accepted	Needs evidence / Draft ready / Submitted / Won / Lost / Auto-accepted	red-or-amber by urgency / blue / blue / green / red / gray
Readiness (server label)	well_defended / moderate / vulnerable	Well-defended / Moderate / Vulnerable	green / amber / red
Evidence quality (server label)	strong / moderate / weak	Strong / Moderate / Weak	green / amber / red
Countdown	over 8h / 4-8h / under 4h / under 1h	hh:mm:ss	green / amber / red / red pulsing
TrustLayer	verified / established / building_trust / not_rated	Verified / Established / Building Trust / Not Yet Rated	green / blue / amber / gray

5.5 Component inventory (build these first, guide phase 1)

Component	Variants and states	Used on
AppShell, Sidebar, TopBar, MobileTabBar	Sidebar badges (unmatched count, open disputes with urgency colour); Live/Test mode pill; collapsed on tablet; bottom tab bar on mobile	All merchant screens
MetricCard	value, delta (up/down/neutral), loading skeleton, error	Dashboard, disputes, customers
StatusBadge	one component, mapped from section 5.4; icon + text + colour	Everywhere
MoneyText	minor units in, currency-aware out; tabular nums; large/regular/compact	Everywhere
PaystackRef	monospace, copy button, link to transaction details	Tables, dispute context
ChannelBadge	card, bank_transfer, ussd, mobile_money, bank icons + label	Transactions, dispute context
CountdownTimer	sizes sm/lg; colour thresholds; pulse; expired; uses server-corrected time	Dashboard banner, dispute list, detail
ReadinessRing	0-100 ring + server label; clickable segments in distribution mode	Dashboard, dispute list, detail
EvidenceChecklist	available / missing / optional rows with suggestion text	Dispute detail
AIDraftEditor	read/edit/regenerate; 'AI-generated, unsubmitted' banner; autosave state; sanitised content	Dispute detail
SubmitStepper	3 steps x pending/in-progress/done/failed with retry	Dispute submission
MatchCard	side-by-side payment vs order, confidence pill, accept / not a match	Reconciliation
DataTable + FilterBar + DateRange	sort, filter, search, pagination, row link, empty/loading/error	Transactions, disputes, customers
FileDropzone	drag-drop, mobile camera capture, progress, remove, type/size errors	Dispute detail, fulfillment
ConfirmModal, Drawer, Toast (Sonner)	destructive variant; drawer for 'Link to order'; toast on save/submit/error	Submit, accept & refund, link order
Skeleton, EmptyState, ErrorState	per-component skeletons; empty with a CTA; error with retry and request id	Every data component
TrustBadge, TrustCard, SignalList	4 states x compact/standard/detailed x light/dark; same component in app preview and widget	TrustLayer, public profile, widget
TrustHistoryChart, RecommendedActions	7/30/90d; reason-coded events; action rows deep-link to the fix	Merchant TrustLayer
EmbedCodeBlock	tabs: Script / iframe / Link / SVG; copy with feedback	Merchant TrustLayer

5.6 Paystack-specific design considerations

- **Currency:** default NGN with the naira sign; the spec also lists GHS, KES and ZAR, so format through `Intl.NumberFormat` with the currency from the API, never a hard-coded symbol. Paystack amounts are in the lowest denomination (kobo/pesewas/cents).
- **References:** show the Paystack reference in monospace with a copy button and never truncate it silently. It is the universal join key, so it links to the transaction details page everywhere it appears.
- **Channels:** card, bank transfer, USSD, mobile money each get an icon and label; do not assume everything is a card payment.
- **Test vs Live:** Paystack has separate test and live modes. Keep the 'Live' pill from your mockup, add a clear 'Test mode' state so nobody mistakes demo data for real money.
- **Dispute language:** use Paystack's terms where merchants know them (dispute, chargeback, evidence, resolve). Show the internal status prominently and Paystack's own status as secondary text.

- **Submission is a chain, not one click:** upload URL, upload file, attach evidence, resolve. Show progress and allow retry per step (Figure 5).
- **Two resolutions:** 'Decline and submit evidence' (primary) and 'Accept dispute / refund' (secondary, destructive, confirmed).
- **Settlement context:** show settlement batch or settlement date on transaction details when the backend provides it; reconciliation defaults to yesterday's settlement period.
- **Brand:** DisputePilot and TrustLayer are third-party products. Do not copy Paystack's logo, colours or checkout styling, and do not imply endorsement or a Paystack-issued badge. If you show a 'Powered by Paystack' mark, follow Paystack's brand guidelines.
- **Connectivity and devices:** many merchants and customers are on mid-range Android phones with variable data. Keep the public pages light, avoid layout shift, and make every merchant action work with one hand on a 390px screen.
- **Time:** show deadlines in Africa/Lagos time with the timezone label; countdowns must use server time to avoid clock-skew errors.
- **Data protection:** Nigeria's data protection law (NDPA 2023) is relevant to any customer data on public surfaces; the safest design is the one in section 8: no personal data publicly. Confirm details with your team's legal contact.

6. Information architecture and flows

6.1 Navigation

Desktop sidebar order follows the workflow: **Dashboard, Transactions, Disputes, Reconciliation, Customers, TrustLayer, Settings**. Add a persistent 'Log fulfillment' quick action (camera icon) because merchants use it right after a delivery. On mobile, use a bottom tab bar: Home, Transactions, Disputes, TrustLayer, More.

Route	Screen	Access	Phase	Priority
/login, /register	Auth + Paystack connect onboarding	Public	1-2	Must
/dashboard	Merchant overview	Auth	2	Must
/transactions, /transactions/[reference]	List, joined detail (transaction to customer to order to fulfillment to dispute)	Auth	3	Must
/disputes, /disputes/[id]	Dispute list; evidence review and submission	Auth	4	Must (flagship)
/trustlayer	Status, signals, actions, history, widget, embed	Auth	5	Must
/t/[slug]	Public TrustLayer profile	Public	5	Must
/embed/[slug] + widget.js	Iframe target + loader	Public	5	Must
/reconciliation	Lean daily workspace (accept AI match, complete)	Auth	3-4	Should (demo path)
Quick-log modal	Fulfillment logger	Auth	4	Should (demo path)
/customers, /customers/[id]	Ledger	Auth	later	Could
/orders	Orders list, create	Auth	later	Could (needed if backend has no seed orders)
/settings	Profile, Paystack connection, notifications, domain verification	Auth	6	Should

6.2 The golden path (build and rehearse this first)

The demo story in your spec, translated into screens. If this path works with realistic mock data on day 8, everything else is polish.

1. **Login / onboarding:** Paystack connected, first sync progress shown.
2. **Dashboard:** metrics, TrustLayer status, reconciliation quick view ('3 need review').
3. **Reconciliation:** AI suggests '87% match' for Order #1032; merchant accepts; completes the session.
4. **Log fulfillment:** quick-log modal, photo, save; readiness and trust signals refresh.
5. **TrustLayer:** status updates; recommended actions shrink; preview shows the customer view.
6. **Customer view:** merchant's product page shows the badge; tap shows the trust card and public profile.
7. **Dispute arrives:** toast + red banner with countdown; open dispute.
8. **Dispute detail:** context already linked; generate/read AI draft; quality 'Strong'; one suggestion.
9. **Review and submit:** review gate, confirmation modal, 3-step progress, success banner, PDF download.
10. **Resolution:** dispute status updates live to Won; TrustLayer history shows the reason-coded change.

7. Screen-by-screen specification

Each screen lists purpose, data, layout, interactions, required states and mobile behaviour. 'Done when' mirrors the guide's definition of done. Endpoint names refer to the backend PDF.

7.1 Login and onboarding

Purpose	Get a merchant from sign-up to first synced data without confusion.
Data	POST /auth/login, /auth/register, /auth/paystack/connect, POST /transactions/sync, GET /auth/me
Layout	Centered card. Onboarding is a 4-step wizard: Paystack key (validates, shows business name) - Notifications - Initial sync (progress bar, 'Syncing 247 transactions...') - First reconciliation prompt.
Interactions	Secret key input is masked with a reveal toggle and never logged or stored client-side; validation happens server-side. Each step can be resumed.
States	Invalid key (inline error, no jargon), sync slow (progress + 'you can leave, we'll notify you'), sync failed (retry).
Mobile	Single column, wizard steps as a top progress bar.

7.2 Dashboard

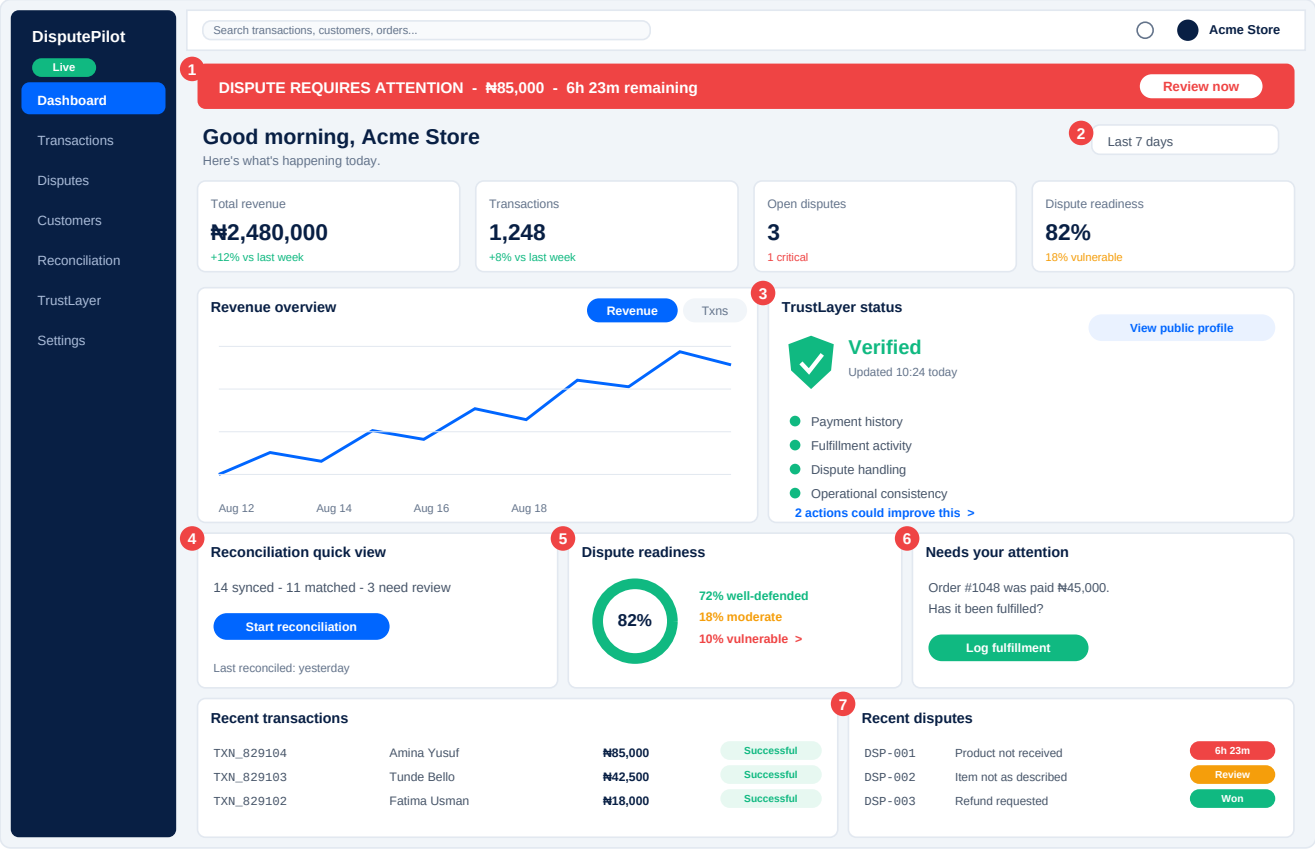


Figure 2. Dashboard (desktop). Numbered markers are explained below.

- **1** Critical dispute banner (missing from the mockup): sticky, not dismissible, live countdown driven by `due_at`.
- **2** Date-range filter (7 / 30 / 90 days) drives every widget on the page.
- **3** TrustLayer status card + count of recommended actions (deep-links to the TrustLayer screen).
- **4** Reconciliation quick view: the daily-habit entry point (spec section 8.1).
- **5** Dispute-readiness distribution; the vulnerable segment is clickable and filters transactions.
- **6** Post-sale nudge: paid order with no fulfillment record, opens the quick-log modal.
- **7** Recent transactions / disputes: every row is a link (reference to transaction details).

Data	GET /dashboard (metrics, trust summary, activity, readiness distribution, action prompts), GET /dashboard/dispute-timeline
Interactions	Date range changes all widgets; metric cards link to filtered lists; the critical banner is sticky and non-dismissible; activity items deep-link.
States	Skeleton per card; empty state for new merchants ('Connect Paystack / sync to see your data'); per-card error with retry so one failure does not blank the page.
Mobile	Single column: banner, 2x2 metric grid, TrustLayer card, quick actions, recent lists (Figure 3).

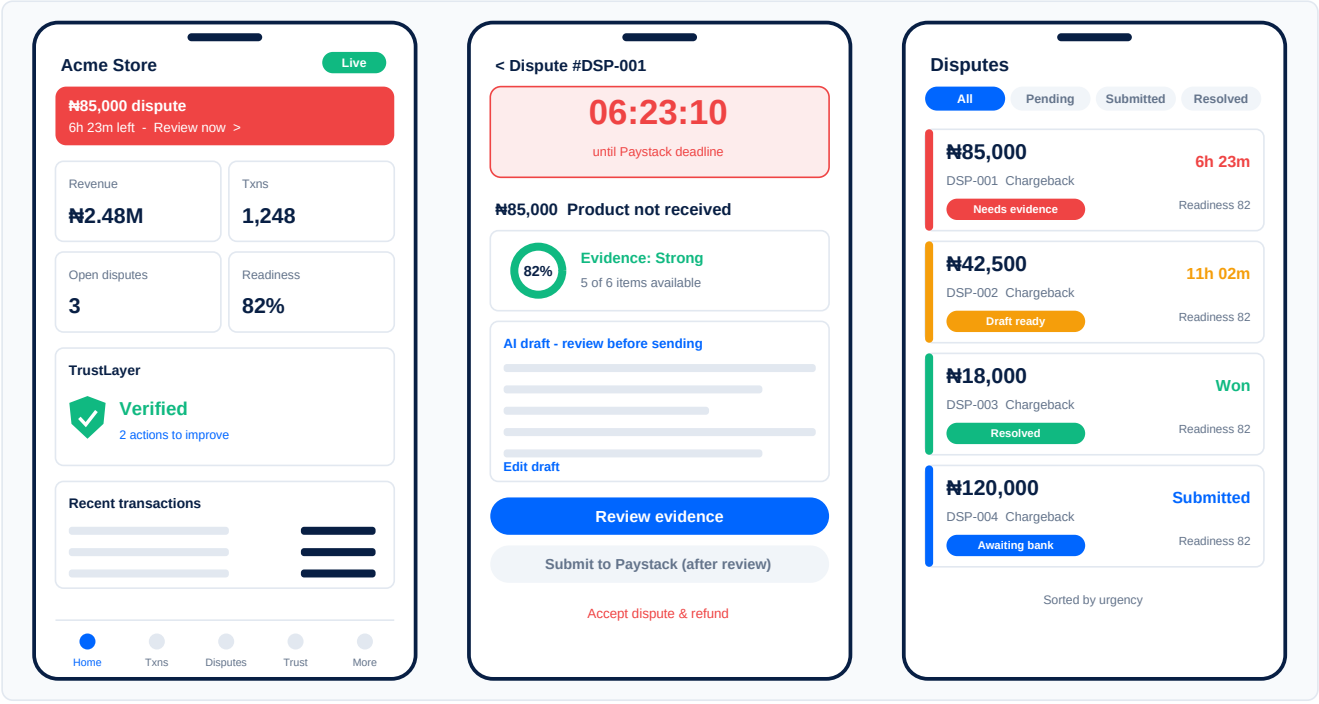


Figure 3. Mobile set: dashboard, dispute detail, dispute list. Bottom tab bar replaces the sidebar; countdown stays above the fold.

7.3 Transactions and transaction details

Purpose	Find any payment and see its full story: transaction, customer, order, fulfillment, dispute and evidence.
Data	GET /transactions (filters: status, reconciliation_status, date range, customer, search), GET /transactions/:reference
Layout	List: search, status/date/channel filters, columns Reference, Customer, Amount, Channel, Status, Reconciliation, Date. Detail: header (amount, status, reference), then a vertical timeline: Payment - Order - Fulfillment - Dispute/Evidence, each a card with a link.
Interactions	Row click opens detail. Detail actions: link to order (drawer), flag/unflag, log fulfillment (quick-log prefilled), open dispute if present.
States	Loading skeleton rows; empty ('No transactions match your filters' + clear filters); error with retry; unmatched transactions show an amber 'Link to order' CTA.
Mobile	Table becomes stacked cards (reference, amount, status, date).

7.4 Disputes list

Purpose	Triage: what needs action first.
Data	GET /disputes (status, category, urgency filters), realtime updates
Layout	Cards sorted by urgency: live countdown, large amount, category badge, customer name, status badge, readiness score. Filter bar: All / Pending / Submitted / Resolved plus date range.
States	Empty is a positive state ('No open disputes. Your evidence is ready if one arrives.') with a link to raise readiness.
Mobile	Same cards, full width; countdown top-right (Figure 3).

7.5 Dispute details (flagship screen)

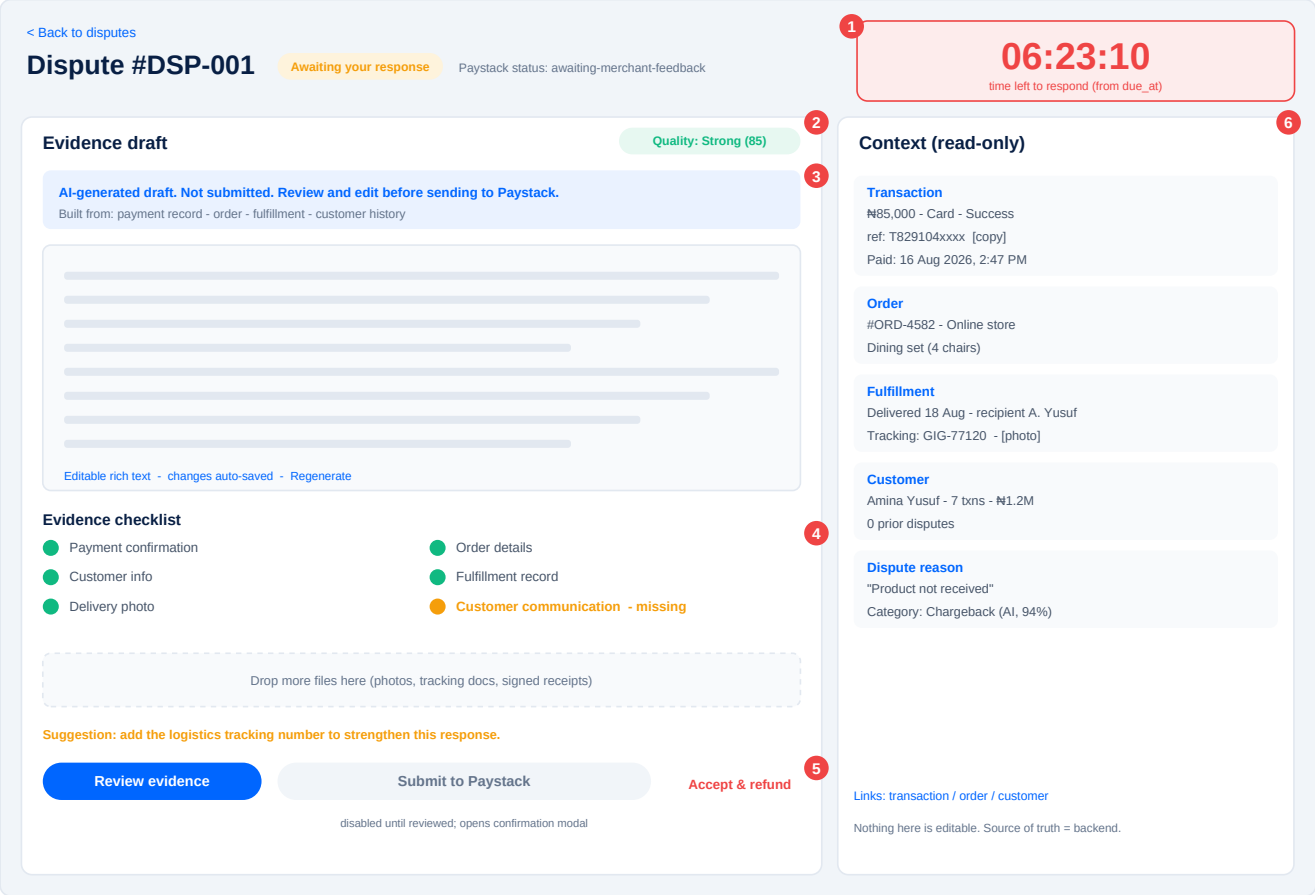


Figure 4. Dispute detail: evidence draft (left) and read-only context (right). Countdown always visible.

- **1** Countdown from server `due_at` (corrected with `server_time`); colour by remaining time: green over 8h, amber 4-8h, red under 4h, pulsing under 1h.
- **2** Quality label comes from the backend (`quality.label`); never hard-code score thresholds in the UI.
- **3** Persistent banner that the draft is AI-generated and unsubmitted (definition of done: never silently submitted).
- **4** Checklist shows exactly which evidence exists and what is missing, with a suggestion for each gap.
- **5** Submit is gated by an explicit Review step + confirmation modal. 'Accept & refund' is a separate, confirmed, destructive path.
- **6** Context panel is read-only and deep-links to transaction, order and customer records.



Figure 5. Submission progress and failure handling.

Data	GET /disputes/:id (joined transaction, order, fulfillment, customer, evidence checklist, quality, files), POST /disputes/:id/generate-evidence, PUT /disputes/:id/evidence, POST /disputes/:id/files, POST /disputes/:id/submit, GET /disputes/:id/evidence-pdf
Interactions	Autosave edits (debounced) with 'Saved' indicator. Regenerate asks for confirmation if edits exist. Submit disabled until evidence exists and the merchant has clicked Review; confirmation modal restates amount, deadline and what will be sent.
Weak evidence	Weak quality shows a red badge and a warning inside the confirmation modal, but does not block submission.
Post-submission	Read-only view, green confirmation banner with time remaining before deadline, PDF download, live status until resolved.
Security	The AI narrative is user-editable text: render as plain text/markdown or sanitise HTML (DOMPurify) before display. Never inject raw HTML.
States	Generating (skeleton + 'Drafting from payment, order and fulfillment...'), generation failed (template fallback banner from backend), expired deadline (read-only + explanation), already resolved.
Mobile	Stacked: countdown, readiness, draft, checklist, actions; context becomes collapsible sections below.

7.6 Reconciliation and fulfillment quick-log

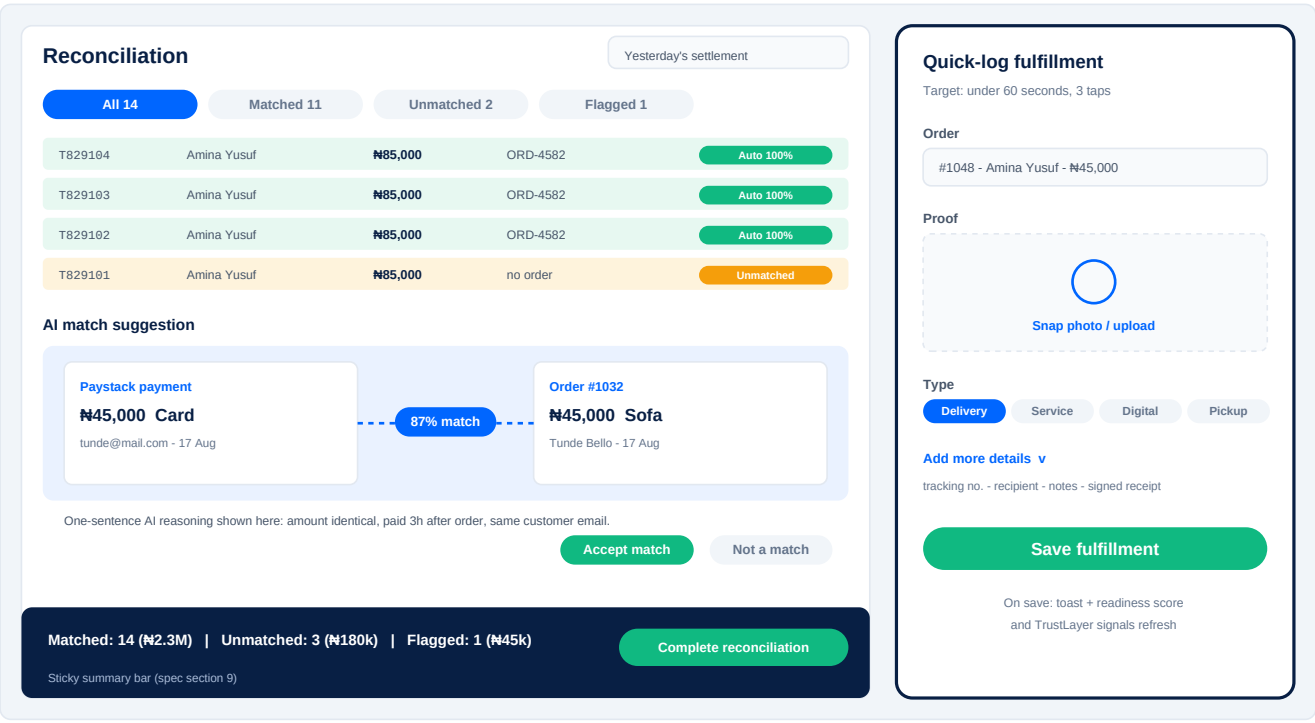


Figure 6. Lean reconciliation workspace with AI match card and sticky summary bar (left); quick-log fulfillment modal (right).

Data	GET /reconciliation/current, POST /reconciliation/accept-match, /reject-match, /complete, GET /orders?search=, POST /fulfillments (multipart)
Interactions	Accept moves the row to Matched with an undo toast. 'Link to order' opens a searchable drawer. Complete triggers the summary notification. Quick-log: select order (search by number, customer, amount), photo, Save; 'Add more details' expands tracking, recipient, notes, signed receipt.

States	Nothing to reconcile (celebratory empty state), sync in progress, AI suggestions unavailable (rule-based matches only), upload failed (retry, keep the form).
Scope note	For the hackathon build Matched, Unmatched with AI cards, and the summary bar. Skip Flagged detail views and session history until later.

7.7 Customers (optional)

Purpose	Ledger context that feeds evidence: transactions, spend, disputes, timeline.
Data	GET /customers, GET /customers/:id/ledger
Scope	Build only if time permits. The dispute detail already shows a customer mini-ledger, which covers the demo need.

7.8 TrustLayer (merchant side)

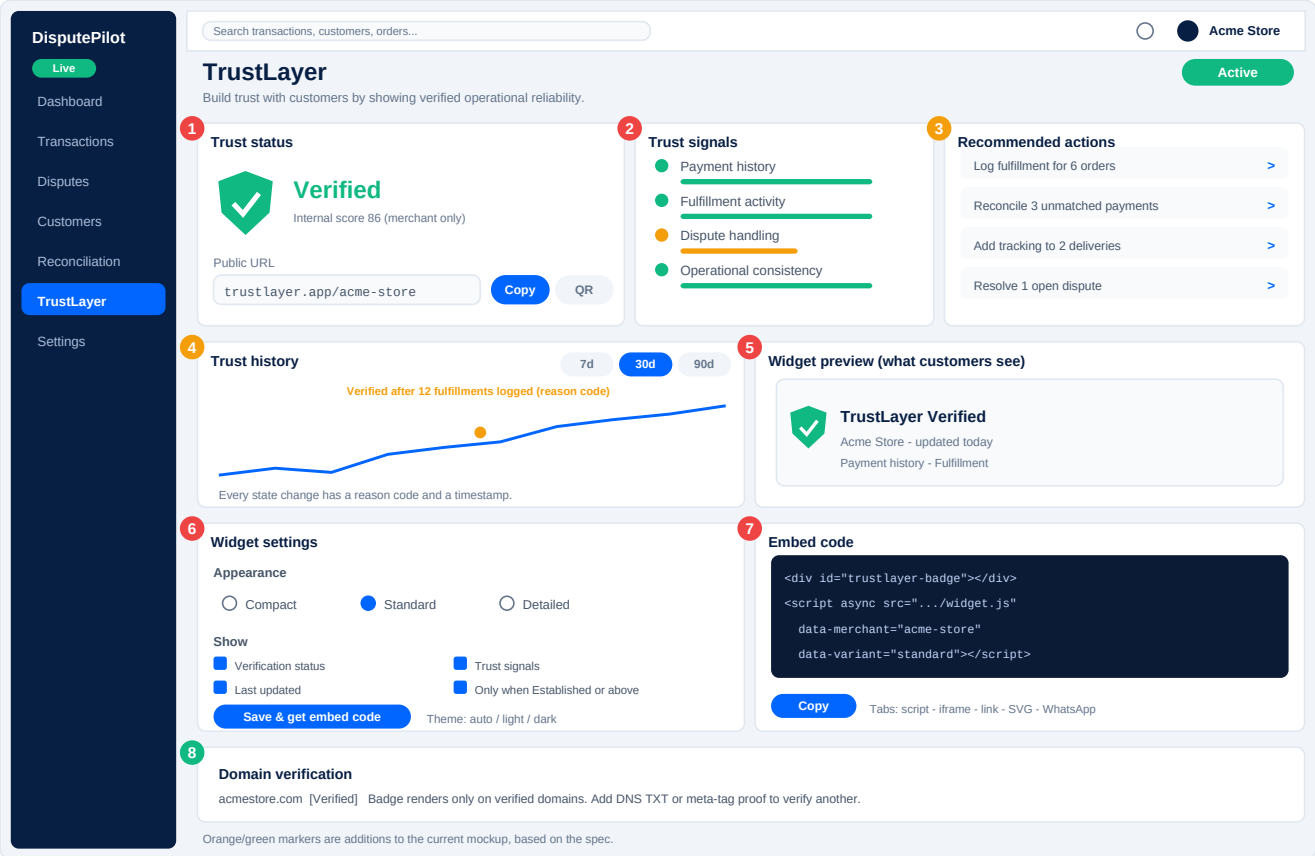


Figure 7. Merchant TrustLayer screen: status, signals, recommended actions, history, live preview, settings, embed and domain verification.

- **1** Public status + share tools (copy link, WhatsApp, QR). Internal score is shown to the merchant only, never public.
- **2** Four signals (spec: 4-5 deterministic signals), each with a plain-language reading, no exact customer counts.
- **3** Recommended actions (in the spec, absent from the mockup): the loop that makes TrustLayer actionable.
- **4** Trust history 7/30/90 days with reason-coded state changes (spec section 13).
- **5** Live widget preview rendered by the same component the customer sees.
- **6** Display options; adds 'only show when Established or above' so a weak state is never forced onto a storefront.
- **7** Embed code with fallbacks for platforms that block scripts.
- **8** Domain verification: the anti-spoofing control. Badge refuses to render on unverified origins.

Data	GET /trust/me, GET /trust/me/history?window=30d, GET /trust/me/actions, GET/PUT /trust/widget/config, POST /trust/recalculate, domain verification endpoints

States	New merchant (Not Yet Rated: explains what unlocks a status), recalculating (spinner on the card, not the page), embed code unavailable until a slug and verified domain exist.
Consistency	The same status must read identically on the dashboard card, this page, the widget and the public page. Use one TrustBadge component fed by one payload.

7.9 Settings

Purpose	Profile, Paystack connection, notifications, domain verification, data export.
Data	GET/PUT /auth/me, POST /auth/paystack/connect, notification preferences, domain endpoints
Rules	Never display the Paystack secret key after saving; show only integration id and connection health. Rotating the key uses a separate confirmed flow.

8. TrustLayer on the customer end

Recommendation
Ship TrustLayer as a three-tier experience: (1) a small domain-verified **badge** on the merchant's own pages, (2) a **trust card** that opens on tap, (3) a hosted **public profile**. Deliver the badge as a script that mounts a sandboxed iframe, and give merchants without a website a shareable link, QR code and WhatsApp/Instagram-ready profile.

8.1 Why this approach

- **Customers never see DisputePilot.** TrustLayer is the public face of private data, so it must be self-contained, safe and instantly understandable.
- **Paystack-hosted checkout cannot be customised** (the architecture doc says so explicitly). The strongest supported moments are on the merchant's own pages: product page, the page just before paying, and the order confirmation.
- **Many merchants sell on WhatsApp and Instagram** (the orders table lists those sources). A hosted profile URL with a QR code is the surface that reaches them; the badge alone would leave them out.
- **A copyable image proves nothing.** A badge that any site can paste is worthless as trust. A live, domain-verified iframe can only show 'Verified' where the merchant proved they own the domain.
- **Progressive disclosure keeps it quiet.** Badge (2 seconds) then card (10 seconds) then profile (1 minute). Customers choose how deep to go, and the checkout flow is never interrupted.

8.2 Where it appears

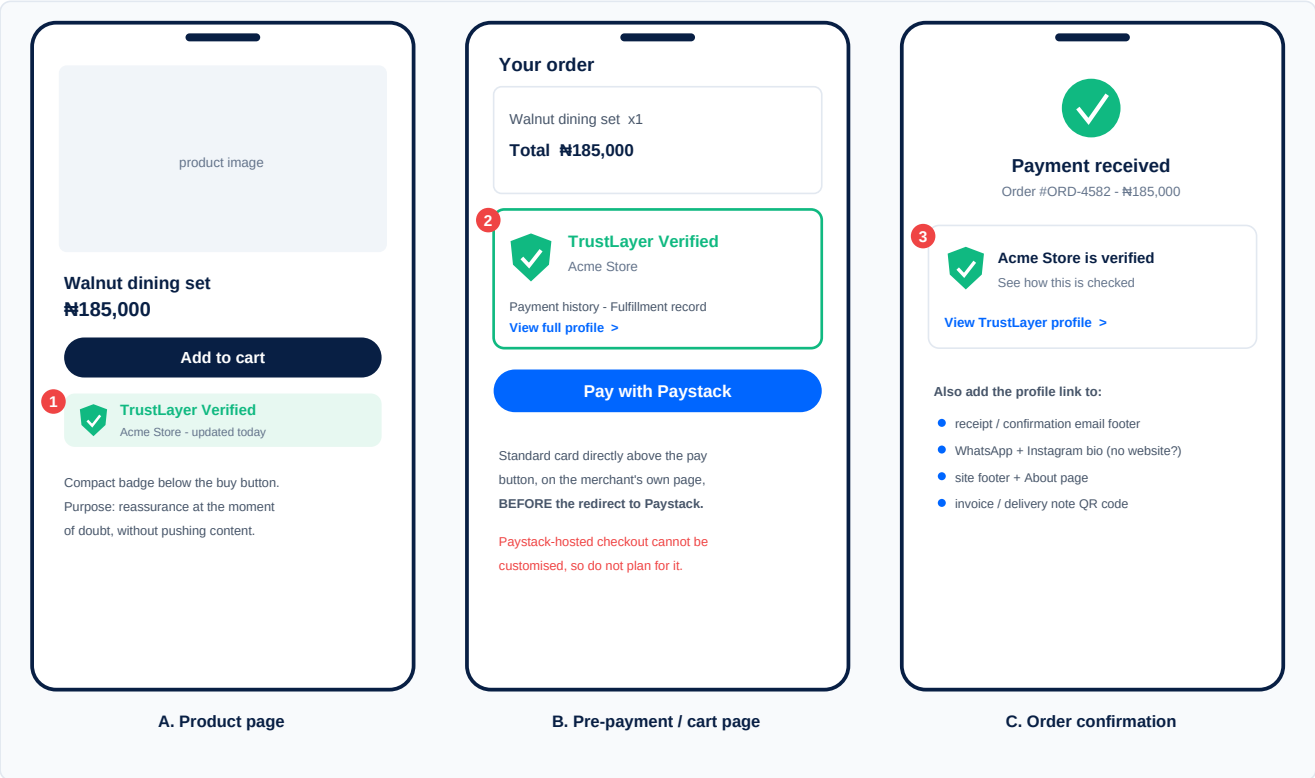


Figure 8. Three placements on merchant-controlled pages. B is the highest-value spot; C extends trust after purchase.

Surface	Variant	Position	Why
Product page	Compact	Directly under Add to cart / Buy	Reassurance at the moment of doubt without adding content
Pre-payment / cart page	Standard card	Above the 'Pay with Paystack' button, before the redirect	Highest intent; customers are about to hand over money

Surface	Variant	Position	Why
Order confirmation	Standard or link	Below the receipt summary	Builds confidence for the next purchase and referrals
Site footer / About	Compact or Detailed	Footer band or About page	Persistent, low-noise presence
Email receipts and invoices	Plain link or SVG badge	Footer, or QR on the delivery note	Scripts do not run in email; the link points to the profile
WhatsApp / Instagram / link-in-bio	Public profile URL + QR	Bio link, catalogue message, status	Reaches merchants who have no website

8.3 Widget sizes, states and behaviour

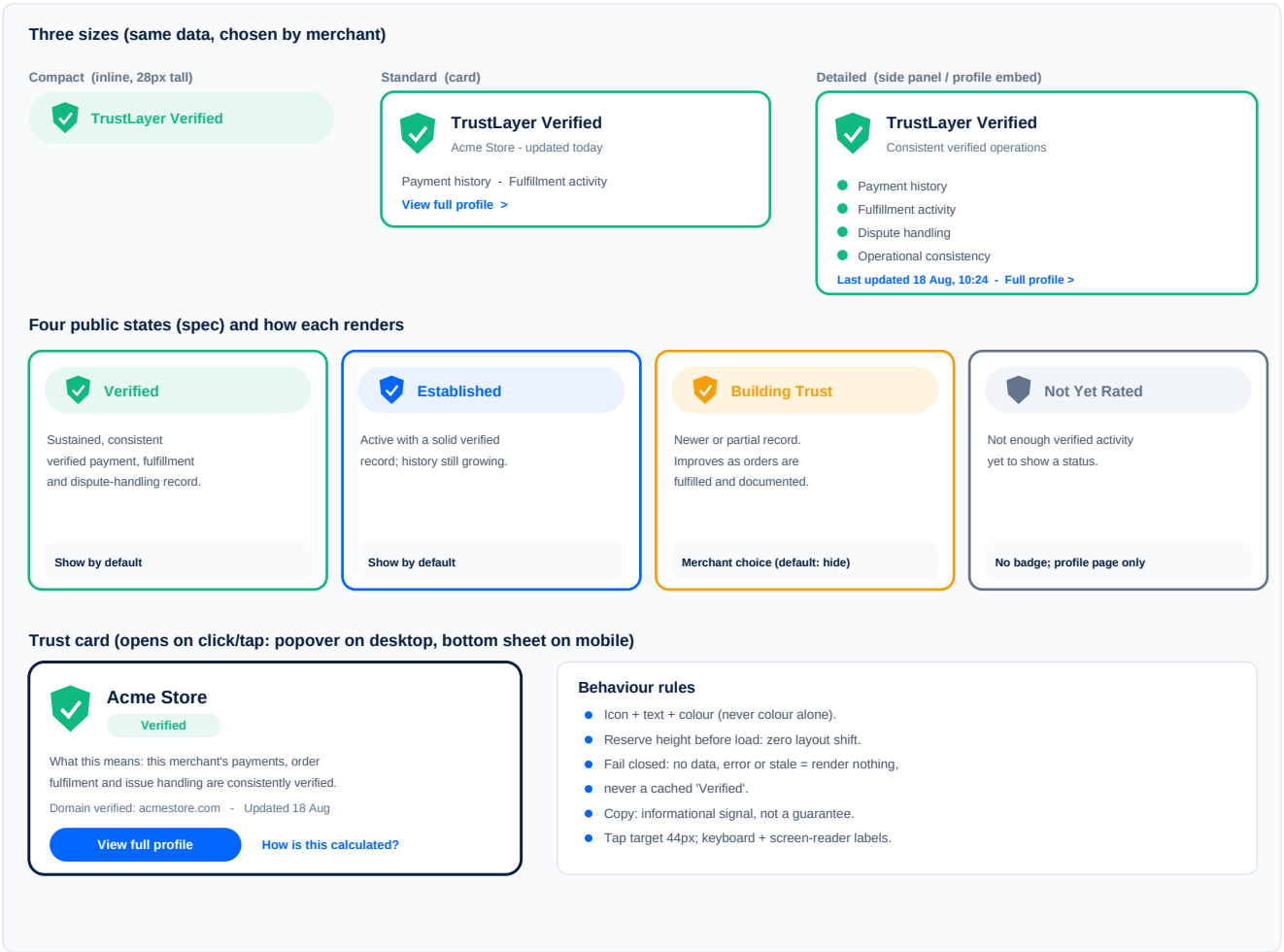


Figure 9. Compact, Standard and Detailed variants; the four public states; the trust card; behaviour rules.

- **Verified** and **Established** show by default. **Building Trust** shows only if the merchant opts in; the default is to hide it on storefronts so a new merchant is not penalised for being new. **Not Yet Rated** never renders a badge; the profile page explains it.
- **Fail closed.** If the request fails, the payload is stale beyond its max age, or the origin is not verified, render *nothing*. Never show a cached 'Verified'.
- **Every public surface shows three things:** status, a one-sentence meaning, and last-updated date. This is the consistency rule from your definition of done.
- **Informational, not a guarantee.** The trust card links to 'How is this calculated?' and carries the disclaimer.
- **Theme:** light, dark and auto (follows the merchant page via prefers-color-scheme); the iframe reads a `theme` attribute.

8.4 The public profile page

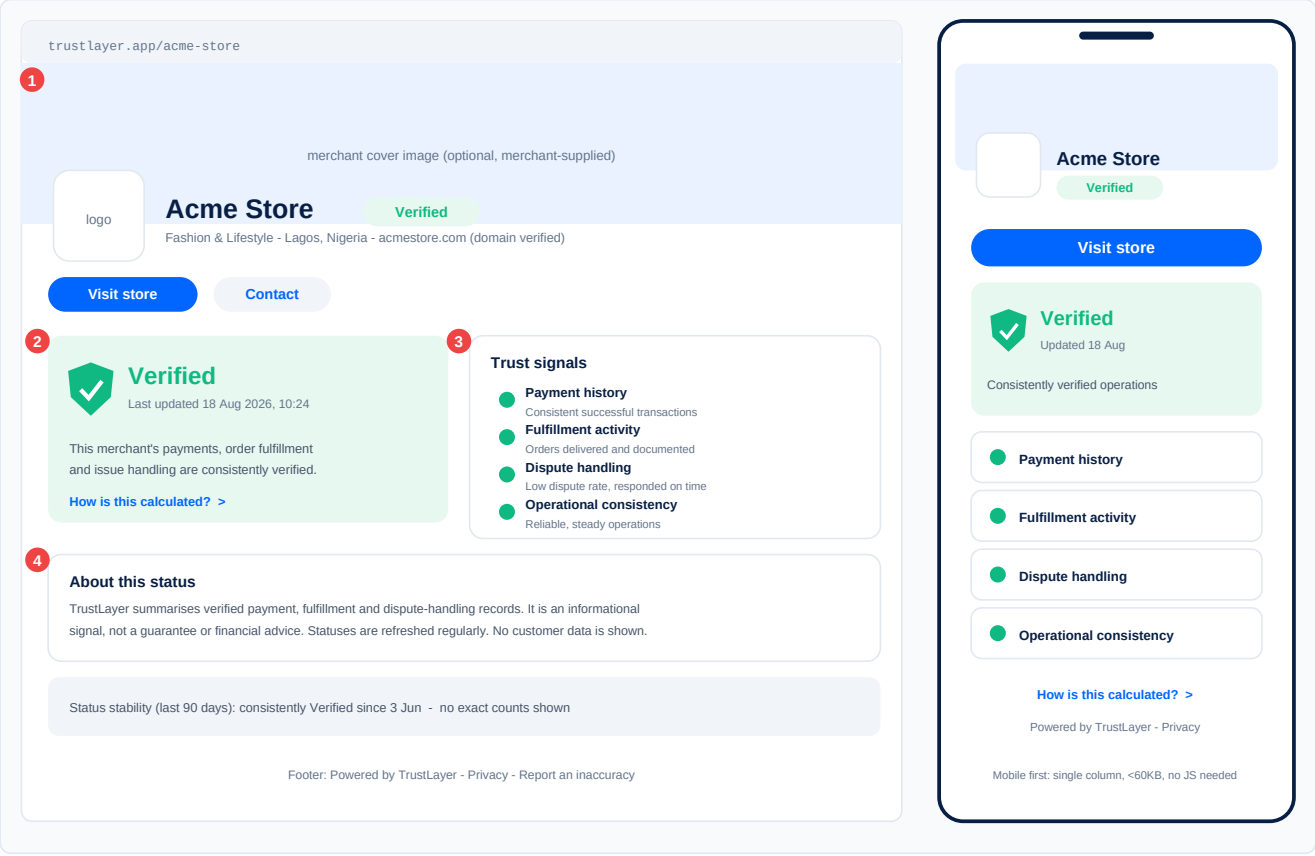


Figure 10. Public TrustLayer profile: desktop and mobile.

- **1** Identity: merchant-supplied name/logo/category; verified domain shown so customers can match it to the site they are on.
- **2** Status + one-sentence meaning + last-updated date (all three are required on every public surface).
- **3** Signals in plain language; qualitative bands only, never raw counts, names or amounts.
- **4** Methodology and disclaimer: informational signal, not a guarantee. Also a 'report an inaccuracy' route.
- **Never on this page:** customer names/emails/phones, individual disputes or evidence, order or amount detail, internal score, backend IDs.

8.5 Embed methods

Method	When to use	Notes
Script loader (recommended): one div + async script with data-merchant, data-variant, data-theme	Any site where the merchant can add HTML (custom sites, Shopify/WooCommerce theme code, WordPress custom HTML)	Loader is about 10KB, creates a sandboxed iframe pointing at /embed/[slug], auto-resizes via postMessage, reserves height to avoid layout shift
Plain iframe	Platforms that strip scripts but allow iframes	Fixed height per variant; no auto-resize
Link + QR	WhatsApp, Instagram, printed materials, delivery notes	QR is generated client-side from the profile URL; includes 'Share on WhatsApp' deep link
SVG badge image (always-current)	Email footers, social posts, marketplaces that only allow images	Served live from TrustLayer, wrapped in a link to the profile; cannot verify the domain, so treat as lower assurance and label accordingly

```
<!-- Recommended embed -->
<div id="trustlayer-badge"></div>
```

```
<script async src="https://cdn.trustlayer.app/widget.js"
        data-merchant="acme-store"
        data-variant="standard"        <!-- compact | standard | detailed -->
        data-theme="auto"></script>    <!-- auto | light | dark -->
```

8.6 Anti-spoofing and safety (design and backend must agree)

Risk	Control	Who builds it
Someone copies a 'Verified' badge onto their own site	Live iframe; merchant registers domain(s); backend serves frame-ancestors only for verified origins; widget shows nothing or 'Unverified placement' elsewhere	Backend + Widget
Stale or forged status	Signed payload, short max-age (about 5 minutes), last-updated visible; widget discards expired payloads	Backend + Widget
Merchant page scripts tamper with the badge	Iframe isolation, sandbox attributes, no access to merchant DOM or cookies	Widget
Merchant site is slowed or broken	Async load, fixed placeholder height, timeouts, silent failure	Widget
Leaking personal data	Public payload has an allow-list of fields (section 8.7); no customer data ever leaves the backend	Backend; frontend never renders more than it receives
Enumeration of merchants	Opaque slugs, rate limiting on public endpoints	Backend

8.7 What the public can and cannot see

Allowed (public)	Never public
Merchant name, logo, category, city/country, verified domain	Customer names, emails, phone numbers, addresses
Public status (one of four) and a one-sentence meaning	Individual transactions, order details, amounts
Four signals as qualitative readings (for example Consistent, Growing, Limited data)	Dispute counts, individual disputes, evidence files, outcomes
Last updated timestamp; status stability over 90 days	Internal numeric score, thresholds, reason codes, event log
Optional banded activity ('500+ verified transactions') only above a minimum sample size	Backend IDs, merchant_id, Paystack IDs or keys

8.8 Copy guide

Use	Avoid (overclaims or unbacked)
TrustLayer Verified	Trusted Merchant, Guaranteed, 100% Safe, Safe to buy
Verified payment and fulfillment record	Reliable Support (no support signal exists), Top rated, 5 stars
Consistent successful transactions	Zero disputes (implies a guarantee; use 'dispute handling')
Updated 18 Aug 2026	Real-time verified (status is periodic, not instant)
An informational signal, not a guarantee	Powered by trusted partners (unbacked)

8.9 Accessibility and performance for the public surface

- Status is never colour alone: icon + text + colour. Text colours meet 4.5:1 (use the text variants from section 5.2).
- Keyboard: badge is a focusable button, the card is a dialog with focus trap and Escape to close; screen readers get 'TrustLayer status: Verified, updated 18 August'.
- Tap targets at least 44px; bottom sheet on mobile, popover on desktop; respects prefers-reduced-motion.
- Budgets: widget loader under 15KB gzip, iframe content under 30KB, public profile under 60KB critical HTML/CSS, LCP under 2.5s on a mid-range Android over 4G, zero layout shift.

- Server-render the profile and cache it (revalidate about every 5 minutes). Ask the backend to trigger on-demand revalidation when a trust state changes.
- Language: plain English now; keep all strings in a dictionary so Pidgin or other languages can be added without code changes.

8.10 Merchant value loop (optional)

Add privacy-safe counters to the merchant TrustLayer screen: badge views, card opens, profile visits. It answers 'is this working?' and motivates merchants to log fulfillments. The backend must aggregate without storing customer identifiers.

Do not build
Injection into Paystack-hosted checkout, customer reviews or star ratings, a public numeric score, or any trust claim without a backing signal. All of these contradict the architecture doc's guidance and weaken the honesty of the demo.

9. Data layer, auth and realtime

9.1 API client and validation

```
// lib/api.ts (shape)
export async function api<T>(path: string, schema: z.ZodType<T>, init?: RequestInit): Promise<T> {
  const res = await fetch('/api/v1' + path, { credentials: 'include', ...init });
  if (!res.ok) throw await toApiError(res);          // { code, message, requestId, status }
  return schema.parse(await res.json());             // fails loudly if the contract drifts
}

// lib/money.ts
export const formatMoney = (minor: number, currency = 'NGN') =>
  new Intl.NumberFormat('en-NG', { style: 'currency', currency, currencyDisplay: 'narrowSymbol' })
    .format(minor / 100);

// lib/countdown.ts  (server-corrected)
export function remainingMs(dueAt: string, serverNow: string, fetchedAtClientMs: number) {
  const skew = Date.parse(serverNow) - fetchedAtClientMs;
  return Date.parse(dueAt) - (Date.now() + skew);
}
```

9.2 Auth

- Access and refresh tokens live in httpOnly, Secure, SameSite cookies set by the Next.js proxy; the browser never reads a token.
- The proxy refreshes silently on 401 once, then redirects to login with a return URL.
- Route guard in the `(app)` layout checks `GET /auth/me`; the public group has no auth and no cookies.
- Paystack secret keys are entered once, sent to the backend, and never stored or echoed by the frontend.

9.3 Caching policy (TanStack Query)

Data	Stale time	Refetch	Notes
Dashboard	30s	On focus + realtime events	Per-widget queries so partial failure is isolated
Transactions list	60s	On focus, after sync	Keep previous data while filtering
Dispute list	15s	Realtime + 30s polling fallback	Urgency changes fast
Dispute detail	0s while open	Realtime + after every mutation	Countdown is client-ticked, not refetched
Evidence draft	Not cached after edit	Mutation returns the new draft	Debounced autosave
Trust (merchant)	60s	After fulfillment/match/dispute mutations	Invalidate trust keys when those mutate
Public trust	Server cache ~5 min	On-demand revalidation	No client polling

9.4 Realtime events the UI listens for

Event	UI effect
dispute.created	Toast + red banner + badge on Disputes; invalidate dispute list and dashboard
dispute.updated / dispute.resolved	Update status in place; if open, show resolution banner; invalidate trust
deadline.warning (8h, 4h)	Escalate banner colour; toast
transaction.synced / reconciliation.updated	Refresh counts and quick view
evidence.ready	Toast with 'Review draft' link
trust.updated	Refresh TrustLayer card; show reason-coded change

If the WebSocket drops, fall back to polling and show a small 'Reconnecting' indicator; never let the countdown freeze silently.

9.5 Error handling

Error	UI
401	Silent refresh once; then login with return URL
403 / 404	Friendly page, no data leak about whether a record exists for another merchant
422 validation	Field-level messages from the API's `errors` map
429	Toast with retry-after; disable the action briefly
5xx / network	ErrorState with retry and the request id for support
Contract mismatch (Zod fails)	Log to console/monitoring with the path; show a generic error, never a broken half-render

9.6 Mock-first development

- Write the Zod schemas from the agreed response shapes on day 1-2; generate TypeScript types with `z.infer`.
- MSW handlers return three seeded merchants: **strong** (Verified, 82% readiness), **incomplete** (Building Trust, missing fulfillments), **new** (Not Yet Rated). These match the demo scenarios in the spec.
- Include awkward data: very long names, missing optional fields, amounts over 1 million, a dispute due in 20 minutes, an expired dispute, an empty list.
- Switch a screen to the live API by removing its MSW handler; the Zod schema will tell you immediately if the shape differs.

10. Quality: states, accessibility, testing

10.1 State matrix (every API-driven screen)

Screen	Loading	Empty	Error	Success	Mobile
Dashboard	Card skeletons	New merchant: connect + sync	Per-card retry	Live data	Stacked
Transactions	Row skeletons	No matches + clear filters	Retry	List	Cards
Transaction detail	Timeline skeleton	Missing links shown as CTAs	Retry / not found	Full timeline	Stacked
Disputes	Card skeletons	'No open disputes' positive state	Retry	Sorted cards	Full width
Dispute detail	Split skeleton; drafting state	No evidence yet: generate CTA	Generation failed, submit failed (step-level)	Submitted banner + PDF	Stacked
Reconciliation	List skeleton	All matched	Retry; AI unavailable	Summary bar	Stacked cards
TrustLayer	Card skeletons	Not Yet Rated + unlock guidance	Retry; recalculating	Status + preview	Stacked
Public profile / widget	Reserved height	Not Yet Rated page / widget hidden	Widget hidden (fail closed)	Status	Mobile-first

10.2 Accessibility checklist

- Semantic landmarks, one h1 per page, visible focus rings, full keyboard operation of tables, modals, popovers and the dropzone.
- Colour contrast 4.5:1 for text, 3:1 for icons/borders; status never colour alone.
- Countdown announces changes politely (aria-live) only at thresholds (8h, 4h, 1h), not every second.
- Modals trap focus and return it on close; toasts are announced; forms have labels and error text tied by aria-describedby.
- Test with keyboard only and a screen reader on the dispute detail and the trust card.

10.3 Testing

Level	Scope
Unit (Vitest)	money, countdown, status mappers, query-key factory, Zod schemas against fixtures
Component (Testing Library)	StatusBadge, CountdownTimer thresholds, EvidenceChecklist, SubmitStepper transitions, TrustBadge four states x three variants
E2E (Playwright)	One demo-path test: dashboard, reconciliation accept, log fulfillment, dispute, submit, trust change; plus the widget mounted on a static HTML page
Visual / manual	Desktop 1440 and mobile 390 screenshots; slow-3G and low-CPU throttling for the public profile and widget

10.4 Security checklist

- No secrets in the client bundle or environment variables prefixed for the browser.
- Sanitise every AI-generated or user-entered rich text before rendering.
- Content-Security-Policy on both apps; public site has no cookies and no third-party scripts.
- CSRF protection for cookie-authenticated mutations (SameSite plus a CSRF token or custom header).
- Confirmation modals for irreversible actions: submit evidence, accept and refund, rotate key, disconnect Paystack.

10.5 Definition of done (extended from your guide)

- Every core screen has desktop and mobile layouts and loading, empty, error, success states.
- A transaction reference leads to its transaction details from every place it appears.
- Dispute details clearly show what evidence exists and what is missing; AI evidence is reviewable and never silently submitted.
- TrustLayer public content exposes no customer PII or private dispute evidence; status is identical across dashboard, preview, widget and public page.
- No UI claim exists that the backend data does not support (see the copy guide).
- Components are reusable; no duplicated badges, money formatters or countdowns.
- The full demo runs merchant dashboard, reconciliation, fulfillment, dispute, evidence, TrustLayer without a dead end, on desktop and phone.

11. Schedule, scope and risks

11.1 Scope by priority

Priority	Items
Must	Design system and shell; dashboard; transactions + detail; disputes list + detail with countdown, AI draft review and submission; TrustLayer merchant screen; public profile; widget; loading/empty/error states; mobile layouts
Should	Lean reconciliation (accept AI match); quick-log fulfillment; onboarding; settings + domain verification; realtime alerts
Could	Customers and ledger; orders CRUD UI; trust history chart polish; badge analytics; dark mode; PWA install
Won't (now)	Injection into Paystack checkout; reviews/ratings; public numeric score; multi-language UI; native apps

11.2 Day-by-day (aligned with the 14-day plan)

Day	Frontend deliverable	Depends on
1	Repo, Next.js, Tailwind, shadcn, tokens, Figma foundations, Zod schemas draft; agree API shapes with backend	Backend contract call
2	AppShell (sidebar, topbar, mobile tab bar), auth screens, MSW set-up, base components (Badge, MoneyText, Skeleton, EmptyState)	-
3	Dashboard with mock data; metric cards, chart, activity, readiness	Mock fixtures
4	Transactions list + detail; PaystackRef, ChannelBadge; filters, search	-
5	Reconciliation lean screen + MatchCard; fulfillment quick-log modal	Recon/fulfillment endpoints (mocked)
6	Disputes list, CountdownTimer, urgency sorting; dispute detail layout	Dispute shape incl. due_at
7	Evidence review: AIDraftEditor, checklist, dropzone, quality badge, submit gate, SubmitStepper, confirmation	Evidence endpoints
8	TrustLayer merchant screen: status, signals, actions, history, widget settings, embed code	Trust profile/config shapes
9	Public profile (SSR/ISR) + widget loader/iframe + TrustCard; placement demo page	Public trust endpoint
10	Integration pass 1: dashboard, transactions, disputes on live API; error taxonomy	Backend stable
11	Integration pass 2: reconciliation, trust, uploads, realtime; fix contract drift	Trust engine live
12	Responsive + accessibility + performance pass; state matrix complete	-
13	Seed scenarios (strong / incomplete / new); rehearse demo path; Playwright demo test	Seeded staging
14	Demo day: no new code. Have a recorded fallback and a mock-mode switch ready	-

11.3 Risks and mitigations

Risk	Mitigation
Backend shapes change late	Zod schemas + MSW; agree a versioned OpenAPI file by day 2; contract test in CI
Realtime not ready	Polling fallback already designed; countdown works without realtime
AI slow or failing during demo	Generating state, backend template fallback, pre-seeded strong draft
Scope creep (customers, dark mode)	Feature flags; hide unfinished nav items
Widget blocked on merchant site	Fallback methods (iframe, link, SVG); test on a static page and a sample Shopify-style theme
Clock skew breaks countdown	Server time in every dispute payload; skew correction in remainingMs
Demo Wi-Fi	Local mock mode; screen recording of the golden path

12. Figma handoff kit

Honest note about Figma
I cannot open or draw in Figma from this chat, so the design exists here as a specification plus wireframes rather than a Figma file. Everything below is written so you can rebuild it in Figma quickly, and so a designer or a connected Figma tool could do it without further questions.

12.1 File structure

Page	Contents
00 Cover and notes	Project, status legend (Draft / Ready for dev), links to this spec
01 Foundations	Variables (colour, spacing, radius, type) with Light and Dark modes; grid and breakpoints; icon set (Lucide)
02 Components	Every component in section 5.5 as a component set with variants and states, on auto-layout
03 Merchant - Desktop 1440	Login, onboarding, dashboard, transactions, transaction detail, disputes, dispute detail, reconciliation, quick-log, customers, TrustLayer, settings
04 Merchant - Mobile 390	Dashboard, transactions, disputes, dispute detail, quick-log, TrustLayer
05 TrustLayer - Customer	Badge variants x states, trust card (popover + bottom sheet), public profile desktop + mobile, placement mockups (product, pre-payment, confirmation)
06 States	Loading, empty, error and success frames for each screen (section 10.1)
07 Prototype flows	Golden path (section 6.2) plus the customer trust journey

12.2 Rules for the file

- Variables named by meaning (`color/status/success/text`), not by hue. One source of truth for Tailwind config.
- 8-pt grid, auto-layout everywhere, components with props named to match React props (`variant`, `size`, `state`).
- Component sets: StatusBadge (type x value), CountdownTimer (state), TrustBadge (state x variant x theme), DataTable row (default, hover, loading).
- Use realistic data: naira amounts, Nigerian names and cities, real Paystack-style references, long names and edge cases.
- Add the numbered annotations from the wireframes to the frames so developers see the intent.
- Prototype only the golden path and the customer trust journey; do not over-prototype.

12.3 Key frames to design first (in this order)

1. Dispute detail with countdown, AI draft and checklist (the flagship).
2. TrustLayer badge: four states, three variants, light and dark.
3. Dashboard with the critical banner.
4. Public profile, mobile first.
5. Merchant TrustLayer screen with actions and history.
6. Reconciliation with the AI match card and the quick-log modal.
7. Mobile dashboard and dispute list.

12.4 Immediate next steps

1. Send the backend PDF to your backend developer and schedule a 30-minute contract call (target: day 1 or 2).
2. Create the repo, tokens and component shells (section 4, 5).
3. Decide the open items in section 2 with the team, especially thresholds, trust endpoints and reconciliation scope.

4. Build the golden path with mocks; connect one screen at a time (guide phase 6).
5. Keep the wireframes in this document as the acceptance reference for each screen.